

Week 4: Attention

Week 4 asks how a model can **re-read** the relevant parts of a sequence instead of squeezing everything through one recurrent state.

- Focus: bottleneck of seq2seq, score $\rightarrow$ softmax $\rightarrow$ weighted sum, Q/K/V, self-attention, positional information
- Output: students should be able to explain attention as query-dependent weighted retrieval
- Bridge sentence: attention changes sequence modeling from compressed memory to direct token-to-token access

Why Week 4 Exists

Week 4 asks how models can revisit relevant tokens directly instead of compressing them through one state.

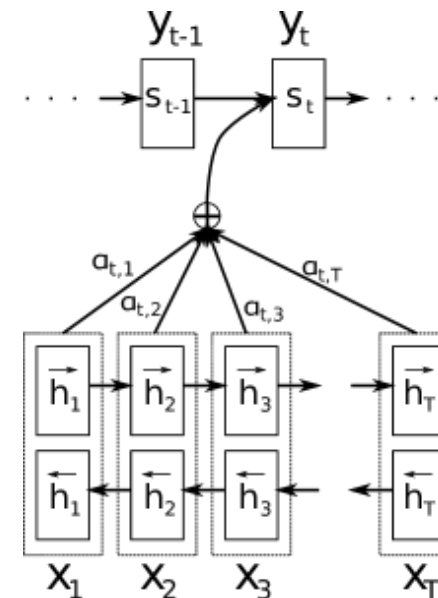
Bridge across weeks

Week	Question	Limitation that remains
3	How can a model carry sequence information forward?	one hidden-state path compresses too much
4	How can a model revisit relevant tokens directly?	needs a relevance computation
5	How does this become a full architecture?	attention needs positional and block-level structure

Seq2Seq Bottleneck

Encoder-decoder models struggled when the decoder had to depend on one fixed summary vector for a long source sequence.

Bahdanau et al. 2015, Fig. 1: attention-based encoder-decoder

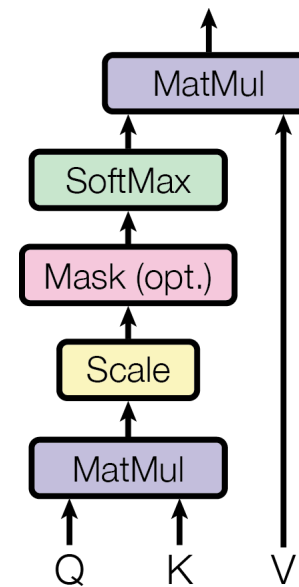


- Classic seq2seq compresses the source into a fixed representation before decoding
- Bahdanau attention lets each decoder step build its own context from source annotations
- Attention relaxes the bottleneck by making source access query-dependent

Attention in Three Steps

At a high level, attention computes relevance scores, normalizes them into weights, and uses those weights to retrieve a context vector.

Vaswani et al. 2017, Fig. 2 left: scaled dot-product attention



- Score: compare each query with available keys
- Softmax: normalize scores into a distribution per query row
- Weighted sum: retrieve values according to those weights

Context Vector Is Query-Specific

Attention does not compute one universal summary of the source; it recomputes context depending on the current query.

- If the query changes, the attention scores change
- If the scores change, the weight distribution over values changes
- Therefore the context vector is a dynamic summary, not a fixed encoding

What Query, Key, and Value Mean

Q/K/V separate the roles of asking for information, matching locations, and retrieving content.

Q/K/V roles

Symbol	Role	Plain-language intuition
Q	query	what information do I want right now?
K	key	how should each token advertise itself for matching?
V	value	what actual content should be returned if matched?

Tensor Shapes of Attention

Attention is conceptually a retrieval mechanism and computationally a sequence of matrix multiplications.

Shape bookkeeping

Missing `\end{aligned}`

```

\ K &:
batch,  $n_k, d_k$ 
\ V &:
batch,  $n_k, d_v$ 
\  $QK^T$  &:
batch,  $n_q, n_k$ 
\  $\text{softmax}(QK^T)V$  &:
batch,  $n_q, d_v$ 
\end{aligned}

```

- Rows of the score matrix correspond to queries
- Columns correspond to keys available for matching
- The final context sequence keeps one representation per query position

Scaled Dot-Product Attention

Dot products are efficient, but we divide by $\sqrt{d_k}$ so the score scale does not become too sharp as dimensions grow.

Core formula

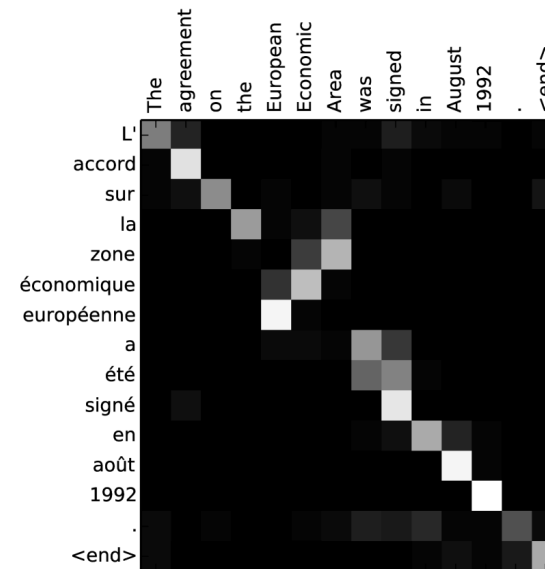
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Large unscaled dot products can make softmax overly peaky
- Scaling stabilizes score magnitude before normalization
- The formula is small, but each term maps cleanly onto a role students already know

Reading an Attention Matrix

The attention heatmap is a relation map: each row asks which source positions a query token consults.

Bahdanau et al. 2015, Fig. 3: learned soft alignment



- Rows/columns should be read as query-target and source-key positions
- Dark cells indicate where a decoder step placed high probability mass
- The plot is useful evidence, but not a complete causal explanation

Encoder-Decoder vs Self-Attention

The same attention mechanism can either relate one sequence to another or let one sequence relate to itself.

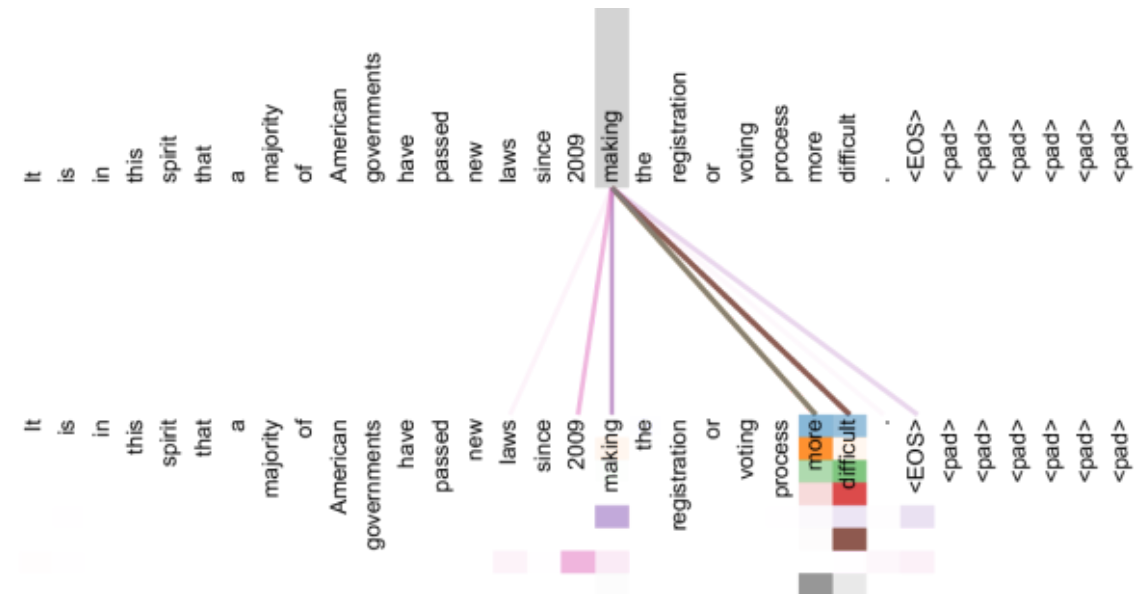
Two attention views

Variant	Where queries come from	Where keys/values come from
encoder-decoder attention	decoder state	encoder outputs
self-attention	the same sequence	the same sequence

Self-Attention as All-Pairs

In self-attention, every token can directly compute relevance with every other token in the same sequence.

Vaswani et al. 2017, Fig. 3: self-attention dependency



- This creates direct paths for long-range dependencies
- It removes the need to funnel everything through one recurrent hidden state
- It also opens the door to more parallel computation than classic recurrence

Why Position Must Be Added

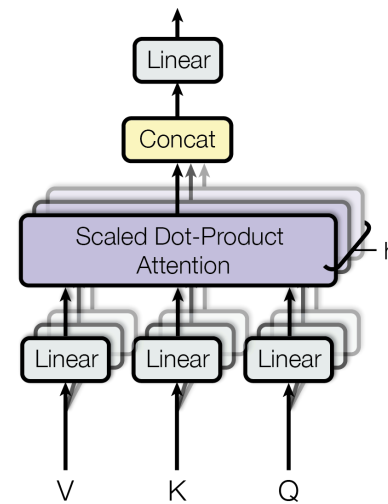
Self-attention relates tokens well, but by itself it does not guarantee that the model knows what came first or later.

- Without a positional signal, token order can become ambiguous
- A model may compare token content correctly while still missing order-sensitive meaning
- Week 5 will turn this need into positional encoding inside the Transformer block

Multi-Head Attention Preview

One attention map may not capture every useful relation, so multiple learned heads can inspect the sequence from different projection spaces.

Vaswani et al. 2017, Fig. 2 right: multi-head attention

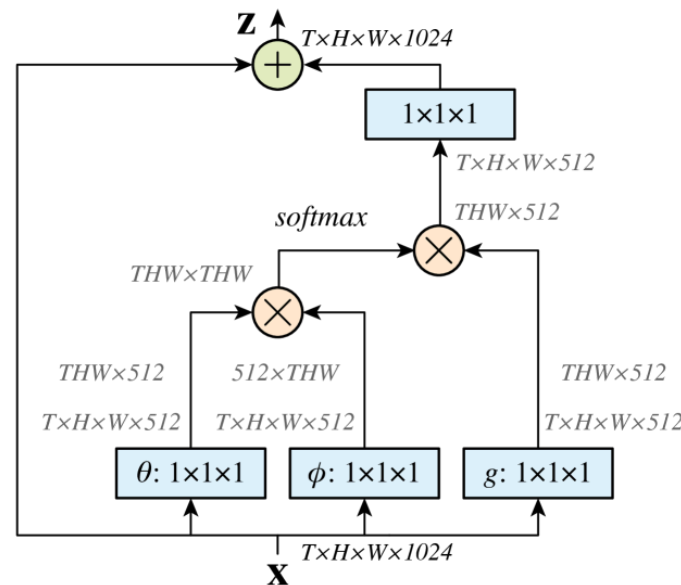


- One head may focus on local proximity, another on long-range agreement
- Different heads can specialize in complementary relation patterns
- The full architectural use of multi-head attention arrives in Week 5

RNN vs Attention: Access Pattern

The real contrast is not old versus new, but compressed sequential memory versus direct weighted retrieval.

Wang et al. 2018, Fig. 2: non-local block



- The same all-pairs weighted-sum pattern appears beyond NLP
- For video/vision, positions can mean space-time locations rather than words
- This echoes the 2025-2 CS231n bridge from RNN temporal modeling to self-attention

What Attention Weights Can Show

Attention weights are valuable probes, but they should not be treated as a complete causal explanation of model reasoning.

- Useful: inspecting relation patterns, qualitative debugging, spotting local vs distant focus
- Not sufficient: proving the entire reasoning process of the model
- Good research habit: combine attention plots with error cases and controlled comparisons

Minimal Tensor Implementation

The clearest way to demystify attention is to implement it once with explicit tensor operations.

Scaled dot-product attention

```
import math

scores = Q @ K.transpose(-2, -1) / math.sqrt(d_k)
weights = torch.softmax(scores, dim=-1)
context = weights @ V
```

- **scores**: relevance matrix between every query and key
- **weights**: normalized distribution per query row
- **context**: weighted combination of values returned to each query position

Optional Preview: Causal Mask

If a model generates text autoregressively, it must prevent each position from looking into the future.

- A causal mask blocks attention to future positions
- This is essential for decoder-only generation models
- We do not need the full implementation today, but students should know why the constraint exists

Checkpoint to Transformer

By the end of Week 4, students should be ready to read self-attention as the engine inside a larger Transformer block.

- Attention computes value-weighted retrieval based on query-key relevance.
- Self-attention lets tokens interact directly across the same sequence.
- Order still needs an extra positional signal.
- The next step is to place self-attention inside a full architecture with multi-head structure, residual paths, and normalization.

